SIMATS ENGINEERING

CONTINOUS INTERNAL ASSESSMENT- ANALYTICAL PROBLEM SOLVING

COURSE NAME: DATABASE MANAGEMENT SYSTEMS
COURSE CODE: CSA0508

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Questions:05 **Total marks:100**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I AKSHAYA SEKAR (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: AKSHAYA SEKAR

1. RELATIONAL ALGEBRA OPERATIONS

Assume the relations EMPLOYEE (Emp_ID, Emp_Name, Salary, Dept_ID) and DEPARTMENT (Dept_ID, Dept_Name, Location). Relational algebra operations can be used to retrieve, combine, and compare records from these relations.

a) Selection (σ): Employees belonging to a particular department

σ Dept_ID = 10 (EMPLOYEE)

This returns all employees whose Dept_ID is 10. Replace 10 with the required department ID.

b) Projection (π): Department details

π Dept_ID, Dept_Name, Location (DEPARTMENT)

This displays only the department ID, department name, and location attributes.

c) Union (∪): Employees in either of two departments

π Emp_ID, Emp_Name (σ Dept_ID = 10 (EMPLOYEE)) ∪ π Emp_ID, Emp_Name (σ Dept_ID = 20 (EMPLOYEE))

Union combines tuples from two union-compatible relations and removes duplicate tuples. Both relations must have the same number of attributes with compatible domains.

d) Set Difference (−): Employees in department 10 but not department 20

π Emp_ID, Emp_Name (σ Dept_ID = 10 (EMPLOYEE)) − π Emp_ID, Emp_Name (σ Dept_ID = 20 (EMPLOYEE))

Set difference returns tuples present in the first relation but absent from the second. The two relations must be union-compatible.

e) Cartesian Product (×): Pair every employee with every department

EMPLOYEE × DEPARTMENT

This produces every possible employee–department pair. If EMPLOYEE has m rows and DEPARTMENT has n rows, the result contains m × n rows.

f) Join (⋈): Employee and corresponding department information

EMPLOYEE ⋈ EMPLOYEE.Dept_ID = DEPARTMENT.Dept_ID DEPARTMENT

The join combines each employee with the department record whose Dept_ID matches, providing employee names, salaries, department names, and locations.

2. SQL TABLE CREATION WITH CONSTRAINTS

The DEPARTMENT table is created first because STUDENT refers to it through a foreign key.

```
CREATE TABLE DEPARTMENT (DeptID INT PRIMARY KEY, DeptName VARCHAR(100) NOT NULL UNIQUE);
CREATE TABLE STUDENT (SID INT PRIMARY KEY, Name VARCHAR (100) NOT NULL, DOB DATE NOT NULL,DeptID INT NOT NULL, Email VARCHAR (150) UNIUE, Age INT DEFAULT 18 CHECK (Age >= 16), FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID));
```

PRIMARY KEY uniquely identifies each row and does not allow NULL values. NOT NULL requires a value. UNIQUE prevents duplicate values. CHECK enforces a condition on values. DEFAULT supplies a value when one is not provided. FOREIGN KEY ensures that every non-NULL DeptID in STUDENT matches an existing DeptID in DEPARTMENT, maintaining referential integrity.

Note: The Age column is included to demonstrate CHECK and DEFAULT constraints. If the schema must contain only the listed attributes, these constraints can instead be applied to an appropriate existing column according to the required business rules.

3. SQL AGGREGATE FUNCTIONS, GROUP BY AND HAVING

Assume the SALES table contains Sale_ID, Product_ID, Customer_ID, Quantity, Unit_Price, and Sale_Date. The total value of each sale line is Quantity multiplied by Unit_Price.

a) Product-wise sales summary

```
SELECT Product_ID,COUNT (*) AS Number_of_Sales,SUM (Quantity *
Unit_Price) AS Total_Sales, AVG (Quantity * Unit_Price) AS Average_Sale,
MAX (Quantity * Unit_Price) AS Highest_Sale, MIN (Quantity * Unit_Price) AS
Lowest_Sale FROM SALES GROUP BY Product_ID;
```

GROUP BY creates one group for each product. COUNT counts sale records, SUM calculates total sales value, AVG calculates the average sale-line value, and MAX and MIN identify the highest and lowest sale-line values.

b) Products with total sales greater than 50,000

```
SELECT Product_ID, SUM (Quantity * Unit_Price) AS Total_Sales FROM
SALES GROUP BY Product_ID HAVING SUM (Quantity * Unit_Price) > 50000;
```

HAVING filters groups after aggregation and returns only products whose total sales value exceeds 50,000.

c) Products with at least 10 sales records

```
SELECT Product_ID, COUNT (*) AS Number_of_Sales FROM SALES GROUP
BY Product_ID HAVING COUNT (*) >= 10;
```

This query returns products that appear in ten or more sales records.

d) Average quantity sold per product

```
SELECT Product_ID, AVG(Quantity) AS Average_Quantity FROM SALES
GROUP BY Product_ID;
```

This calculates the average quantity recorded per sale for each product.

4. CORRELATED SUBQUERY TO FIND EMPLOYEES ABOVE THEIR DEPARTMENT AVERAGE

```
SELECT E.Emp_ID, E.Emp_Name, E.Salary, E.Dept_ID
FROM EMPLOYEE E WHERE E.Salary > (SELECT AVG(E2.Salary) FROM
EMPLOYEE E2 WHERE E2.Dept_ID = E.Dept_ID
);
```

The outer query examines each employee E. For that employee, the inner query calculates the average salary of employees in the same department by matching E2.Dept_ID with E.Dept_ID. The outer query then returns the employee only when their salary is greater than that department average. Because the inner query refers to the current outer-row department, it is a correlated subquery and is evaluated in relation to each employee row.

The query compares the employee's salary with the average salary of all employees in the same department, including that employee. If the intention is to calculate the average of other employees while excluding the current employee, add the condition `AND E2.Emp_ID <> E.Emp_ID` inside the subquery.

5. SQL JOIN OPERATIONS

Assume EMPLOYEE(Emp_ID, Emp_Name, Dept_ID) and PROJECT (Project_ID, Project_Name, Emp_ID). The common attribute Emp_ID links an employee to a project.

a) INNER JOIN

```
SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name FROM
EMPLOYEE INNER JOIN PROJECT P ON E.Emp_ID = P.Emp_ID;
```

Returns only employees who have a matching project record and projects that match an employee.

b) LEFT OUTER JOIN

```
SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
FROM EMPLOYEE E
LEFT OUTER JOIN PROJECT P ON E.Emp_ID = P.Emp_ID;
```

Returns every employee, including employees with no project. Project columns are NULL when no matching project exists.

c) RIGHT OUTER JOIN

```
SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name FROM
EMPLOYEE E RIGHT OUTER JOIN PROJECT P ON E.Emp_ID = P.Emp_ID;
```

Returns every project, including projects with no matching employee. Employee columns are NULL when no matching employee exists.

d) FULL OUTER JOIN

```
SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name FROM
EMPLOYEE  E FULL OUTER JOIN PROJECT P ON E.Emp_ID = P.Emp_ID;
```

Returns all matched employee–project pairs, all employees without projects, and all projects without matching employees. Unmatched columns are filled with NULL. Some database systems, including MySQL, do not directly support FULL OUTER JOIN; it can be emulated using LEFT JOIN and RIGHT JOIN with UNION.

e) CROSS JOIN

```
SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name FROM
EMPLOYEE  E CROSS JOIN PROJECT P;
```

Returns every possible employee–project combination, regardless of whether an actual assignment exists. If there are m employees and n projects, the result contains m × n rows.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation